Genetic Programming-Based Neural Architecture Search for Next-Word Prediction

Yadlapalli Sai Abhiram, Harish, Prashant, Vinayak, Vishnu, Abhishek
Department of Computer Science and Engineering
Vellore Institute of Technology-AP
Amaravati, Andhra Pradesh, India

***Abstract*—The automatic model structure discovery from neural architecture search (NAS) research shows better results than manual model design but most existing NAS techniques require heavy computational resources and provide limited understanding of their operations. The paper introduces a Genetic Programming-based NAS system (GP-NAS) which performs next-word prediction tasks on the WikiText-2 benchmark. Our method evolves the neural sequence model body through a GP tree structure which consists of GRU, LSTM, Transformer, Attention, and FeedForward processing blocks while keeping the embedding stem and classification head unchanged. The GP discovered 120 different architectural designs through 10 generations which used a two-step evaluation system that first assessed 20 percent of training data before testing the highest 15 candidates on 50 percent of training data.The best discovered architecture, GRUR(TF(LSTMR(ID))), achieved a test perplexity of 155.06 — a 23.4% improvement over the strongest baseline — while using 32% fewer parameters, demonstrating that GP-discovered hybrid architectures outperform individually designed models.**

***Index Terms*—Neural Architecture Search, Genetic Programming, Next-Word Prediction, Language Modeling, WikiText-2, Recurrent Neural Networks, Transformer, DEAP**

I. Introduction

Next-word prediction serves as the central challenge for language modeling which underpins various applications that include autocomplete systems and large-scale generative models. The standard approach to building such systems has historically involved choosing a model family — RNNs, LSTMs, or Transformers - and manually tuning its depth, width, and connectivity. The existing method functions effectively, yet it permits users to construct their own architectural designs because architects tend to select their building materials based on established conventions rather than conducting thorough testing of various options.

The architectural design process of Neural Architecture Search (NAS) needs to be handled as an optimization problem which treats architectural design elements as variables to be optimized. The initial NAS techniques used reinforcement learning [1] and evolutionary strategies [2] to search through architectural design spaces which resulted in effective outcomes that required extensive computational resources reaching over 1000 GPU hours. The DARTS method [3] enables cheaper search processes through its implementation of differentiable search methods but limits search capabilities to its predefined cell designs and provides reduced understanding of its operations.

Genetic Programming (GP) provides an alternative way of looking at things. GP uses expression trees to represent architectural designs which it evolves through the application of selection and crossover and mutation operators that mimic biological processes [4]. The resulting architectures are explicitly structured, interpretable, and unconstrained in topology, making GP a natural fit for architecture search over heterogeneous block types.

The researchers employed GP-based NAS to solve next-word prediction for WikiText-2 [5] which serves as a standard language modeling benchmark. The system develops sequence model body through GP tree evolution which uses differentiable neural blocks as nodes that include GRU and LSTM and Transformer encoder layer and multi-head attention and feedforward block and dropout. The stem (GloVe [6] + POS embedding fusion) and head (linear projection to vocabulary) are fixed across all candidate architectures, ensuring that differences in fitness reflect architectural differences rather than input representation.

The main contributions of this work are as follows:

- A GP-NAS pipeline that evolves sequence models through heterogeneous block combinations, verified through a two-stage proxy-then-full-training strategy.
- Empirical evidence that GP-discovered hybrid LSTM-Transformer architectures outperform pure recurrent and Transformer baselines on WikiText-2 with fewer parameters.
- A detailed analysis of proxy evaluation failure: Spearman rank correlation of 0.021 between proxy and true rankings, revealing systematic bias against Transformer-based architectures in short-epoch evaluations.
- A full characterization of GP population dynamics, including early convergence to recurrent-dominated solutions and the role of random immigrants in maintaining diversity.

II. Related Work

A. Language Modeling and Next-Word Prediction

The process of language modeling requires researchers to predict which sequences of tokens will occur at different probabilities. The first neural systems implemented through

feedforward networks [11] developed into a system that used LSTMs [9] and GRUs [10] as their main method for capturing sequential dependencies. The field experienced a major transformation when attention mechanisms [12] and the Transformer architecture [8] became available because Transformers now serve as the primary foundation for all major language models. WikiText-2 [5] serves as a standard evaluation benchmark in this field which researchers use to measure language model perplexity because it compiles verified Wikipedia articles.

B. Neural Architecture Search

The introduction of NAS provided a systematic method to create automated model design processes. The research conducted by Zoph and Le [1] achieved competitive image classification results through their use of a REINFORCE-trained recurrent controller for architecture description generation, but this method required extreme computational resources. The evolutionary approaches [2] utilized tournament selection combined with weight inheritance to explore cell structures, which proved that evolution could achieve results that matched or surpassed those obtained through RL-based search. DARTS [3] achieved differentiable searching through its development of continuous architectural selection, which allowed researchers to complete their search process within a few GPU days. The architectural design of differentiable methods limits their application to specific cell structures, which makes them less adaptable than tree-based Architectural models.

C. Genetic Programming for Architecture Search

GP has been applied to NAS in several forms. Angeline [13] demonstrated the general applicability of GP to program synthesis and structured optimization. Recent research has utilized GP tree structures to develop both CNN models and recurrent cell designs [14]. The use of DEAP [7] — a Python framework for evolutionary computation — has made GP-based NAS accessible in research settings. GP-based NAS methods provide evaluation of completely separate architectures without needing weight sharing or relaxation, which differs from gradient-based NAS methods.

D. Pretrained Embeddings in Language Models

GloVe [6] embeddings represent words as dense vectors trained on global co-occurrence statistics. The system achieves rapid progress through its initial semantic knowledge which applies to all tasks but shows greater effectiveness in environments with limited resources. Part-of-speech (POS) tags provide complementary syntactic information [15]. The combination of POS features and word embeddings has demonstrated performance enhancements for structured prediction tasks which we include in our embedding stem.

III. Methodology

A. Dataset and Preprocessing

The WikiText-2 dataset [5] is accessed through the HuggingFace `datasets` library. The dataset shows 23,767 training lines 2,461 validation lines and 2,891 test lines after empty lines have been removed, which corresponds to approximately 2M 217K and 245K tokens based on the three datasets.

The text undergoes lowercase conversion and whitespace-based tokenization while maintaining all punctuation marks. The training split is used to create a vocabulary which includes words that appear with a frequency of three or more, resulting in a final vocabulary, which contains 29,573 tokens. Three special tokens are added: `PAD` (index 0), `UNK` (index 1), and `EOS` (index 2).

The system builds sequences by using a sliding window method, which extends for 64 units, while it moves forward by one, which creates shared tokens between all subsequent windows. The sample contains 63 input tokens together with one target token, which represents the 64th token. The dataset generates 1,075,301 training samples and 110,869 validation samples and 124,201 test samples. All data loaders use batch size 64.

B. POS Tagging

Each training line is POS-tagged using spaCy's `en_core_web_sm` model. The system requires individual line tagging because Google Colab experiences a memory problem with `nlp.pipe`. The POS vocabulary contains 20 tags including `PAD_POS` and `UNK_POS` which include all standard Universal Dependencies tags. The system stores POS IDs together with token IDs to create an additional input channel.

C. Embedding Fusion

Every GP-evolved model uses a fixed stem which includes an embedding fusion layer as its main component. The word embeddings start from GloVe 6B 100-dimensional vectors [6], achieving 96.2% vocabulary coverage because 28447 out of 29573 words are included. The 1,126 out-of-vocabulary words receive their initialization through random normal vectors which have a mean value of 0 and a standard deviation of 0.4. POS tags are embedded into a learned 16-dimensional space. Two embeddings merge into a 116-dimensional vector which passes through a fusion layer that uses the following process: `Linear(116→128)` → `LayerNorm(128)` → `ReLU` to create a 128-dimensional representation for all architectures.

GloVe embeddings are frozen for the first three epochs and unfrozen from epoch 4, allowing initial training to stabilize with fixed representations before fine-tuning them to the task.

D. GP Node Definitions

The GP search evolves the *body* of the model — the sequence of processing blocks between the fixed stem and fixed head. Eight block types are defined as GP primitives, each mapping an input tensor of shape $[B, S, 128]$ to an output tensor of the same shape:

- **GRUBlock**: `GRU(128→128)` → `Dropout(0.1)` → `LayerNorm(128)`, optional residual connection.
- **GRU_RBlock**: Same as GRUBlock with `use_residual=True`.

- **LSTMBlock**: `LSTM(128→128)` → `Dropout(0.1)` → `LayerNorm(128)`, optional residual.
- **LSTM_RBlock**: Same as LSTMBlock with `use_residual=True`.
- **TransformerBlock**: Positional encoding → Masked Multi-Head Attention (128d, 4 heads, causal mask) → `LayerNorm` → FeedForward(128→256→128) → `LayerNorm`.
- **AttentionBlock**: Positional encoding → Masked Multi-Head Attention (128d, 4 heads) → `LayerNorm`. No inner feedforward sublayer.
- **FeedForwardBlock**: `Linear(128→256)` → `ReLU` → `Dropout(0.1)` → `Linear(256→128)` → `LayerNorm`, optional residual.
- **DropoutBlock**: `Dropout(0.3)` passthrough — acts as a regularization node.

Two terminals are defined: `x` (the input tensor from the stem) and `ID` (identity passthrough). Each primitive has arity 1, meaning the GP tree represents a sequential function composition.

E. Model Architecture

Every GP-evolved model follows a fixed three-part structure:

$$\hat{y} = \text{Head}(\text{Body}_\theta(\text{Stem}(w, p))) \tag{1}$$

where w is the token input sequence, p is the POS input sequence, *Stem* is the GloVe+POS fusion layer, *Body* is the GP-evolved block sequence, and *Head* is `Linear(128→|V|)` applied to the last timestep only.

A placeholder for the overall system architecture diagram is shown in Fig. 1.

F. GP Configuration

The GP framework is implemented using DEAP [7]. The full configuration is summarized in Table I.

TABLE I
GP HYPERPARAMETER CONFIGURATION

Parameter	Value
Population size	20
Generations	10
Tree min/max depth	2 / 4
Crossover operator	$p_{cx} = 0.5$
Mutation operator	$p_{mut} = 0.3$
Selection	Tournament (size=3)
Elitism	Top 2, from generation 7 onward
Random immigrants	2 per generation (replace worst 2)
Fitness	val_perplexity $+ 0.5 \times \log_{10}$(params)
Fitness type	`FitnessMin`, weight = -1.0
Parameter cap	8M (penalty = 9999 if exceeded)

The fitness function balances predictive quality against model size. Minimizing val_perplexity $+ 0.5 \times \log_{10}$(params) penalizes unnecessarily large models while prioritizing low perplexity.

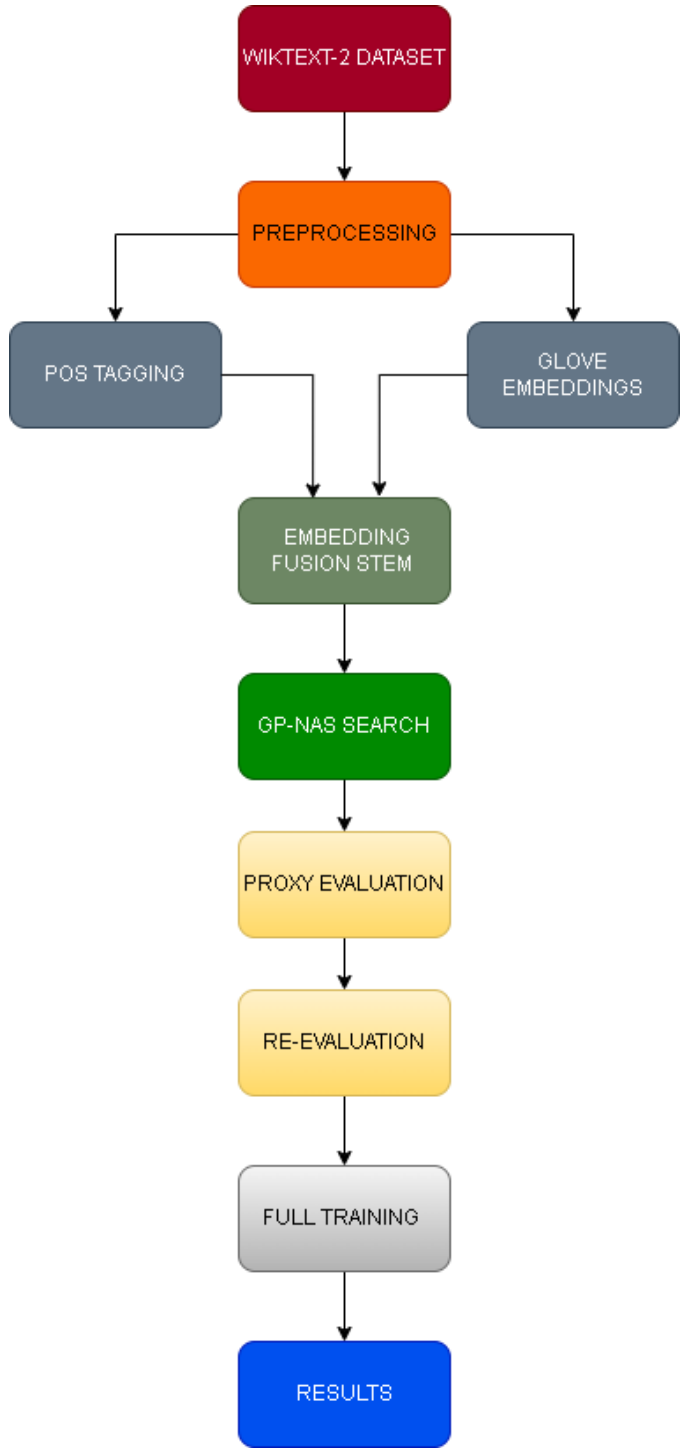

Fig. 1. Overall GP-NAS pipeline: fixed stem and head with GP-evolved body.

Architecture caching is employed to avoid re-evaluating identical tree strings. Over 120 unique architectures were evaluated across the 10 generations.

G. Two-Stage Evaluation Strategy

The GP testing becomes too expensive when we try to check all candidates using the complete dataset. Our two-stage strategy handles this situation.

Stage 1 — Proxy Evaluation: The first five generations show new architectural designs which undergo 2-epoch training tests through 20

Stage 2 — Re-evaluation: After the GP run, the top 15 architectures by proxy fitness are re-evaluated for 5 epochs on 50% of the full training data, using the full validation set. The method solves the problem which occurs when proxy evaluation times get reduced.

Full Training: The complete training set receives 10-epoch training for the top three architectures which achieved highest re-evaluation rankings.

IV. Experiments

A. Hardware and Software

All experiments were conducted on Google Colab using an NVIDIA T4 GPU (15GB VRAM, 12GB RAM). The software stack was: Python 3.10, PyTorch 2.x, CUDA 12.x, DEAP 1.4.1, spaCy 3.x with `en_core_web_sm`, and HuggingFace `datasets`.

B. Full Training Settings

The three selected architectures use the Adam optimizer as their training method which starts with a learning rate of 0.001 and applies gradient clipping with a limit of 1.0. The `ReduceLROnPlateau` scheduler decreases the learning rate by 50 percent after two training intervals of no progress. The system uses early stopping which allows three unsuccessful attempts before it terminates. The loss function uses cross-entropy which is calculated from the final timestep prediction only. GloVe embeddings remain unchanged during the first three epochs but start to be fine-tuned from the fourth epoch. The best checkpoint by validation loss is saved after each epoch. All experiments use seed 42 as the fixed value to ensure reproducible results.

V. Results

A. GP Evolution Dynamics

Fig. 2 shows the best and average fitness scores across all 10 GP generations.

The best fitness drops from 339.20 in generation 1 to 326.24 in generation 2, after which it remains flat through generation 10 (Fig. 2). The best-performing proxy architecture across all generations was `GRU(GRU_R(LSTM_R(ID)))` with a proxy fitness of 326.24. The proxy evaluation approaches this method because it favors rapid architectural convergence which GRU-based systems achieve within 2 to 3 training epochs.

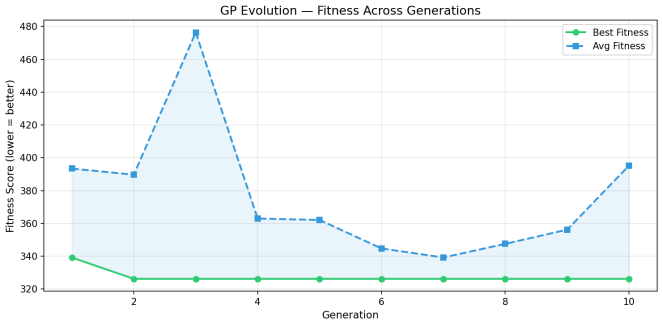

Fig. 2. GP fitness evolution across 10 generations. Best fitness (green, lower is better) converges after generation 2. Average fitness (blue dashed) shows high variance, reflecting diversity maintained by random immigrants.

The average fitness shows considerably more variance, with a spike in generation 3 (avg. 476.42) followed by gradual stabilization. The population experiences two different influences: the random immigrant mechanism and crossover/mutation process, which introduces weaker individuals into the population. The expected behavior in generational GP with diversity preservation leads to this outcome.

B. Proxy Evaluation Failure

Fig. 3 plots the proxy rank against the true re-evaluation rank for the top 15 architectures.

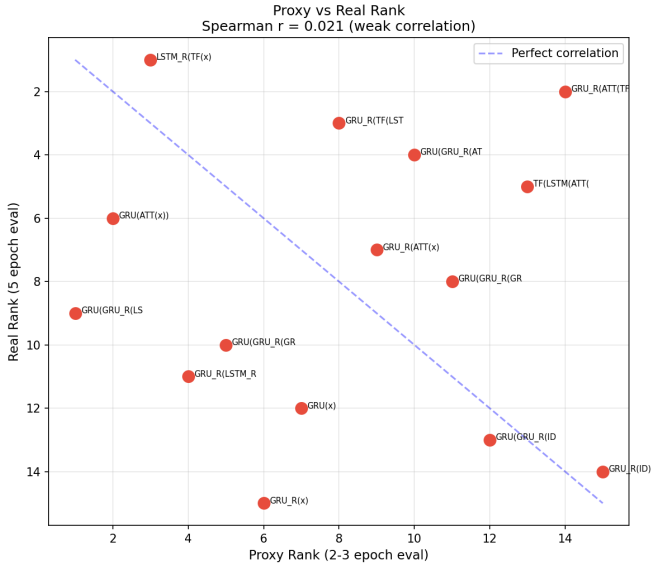

Fig. 3. Proxy rank vs. true re-evaluation rank for the top 15 architectures. Spearman $r = 0.021$, indicating near-zero correlation. Points far from the diagonal represent severe proxy misjudgments.

The Spearman rank correlation between proxy rank and re-evaluation rank is $r = 0.021$ — effectively zero. The architecture ranked 1st by the proxy (`GRU(GRU_R(LSTM_R(ID)))`) ranked 9th in re-evaluation. The architecture that ranked 14th in proxy evaluation (`GRU_R(ATT(TF(LSTM_R(x))))`) ranked 2nd in true

evaluation. This inversion demonstrates that the proxy evaluation was not a reliable predictor of true architectural quality.

C. Re-evaluation of Top 15 Architectures

Fig. 4 shows the re-evaluation perplexity for all 15 architectures ranked by true performance.

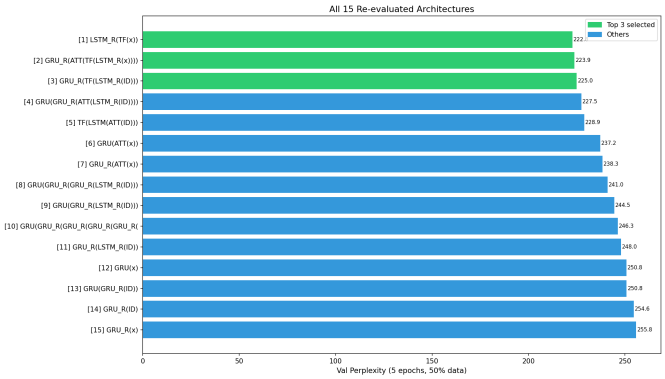

Fig. 4. Re-evaluation perplexity of all 15 candidate architectures (5 epochs, 50% training data). Top 3 selected for full training are shown in green.

The top three architectures selected for full training are:

1) `LSTM_R(TF(x))`: val. perplexity 222.77, params 7.05M
2) `GRU_R(ATT(TF(LSTM_R(x))))`: val. perplexity 223.86, params 7.22M
3) `GRU_R(TF(LSTM_R(ID)))`: val. perplexity 225.02, params 7.15M

All three are hybrid architectures containing at least one Transformer or Attention block. No pure recurrent architecture appeared in the true top 5 of the re-evaluation.

D. Full Training Results

Table II presents the final test results for all models.

TABLE II
FINAL MODEL COMPARISON — TEST SET RESULTS

Model	Test PPL	Top-1 (%)	Top-3 (%)	Params (M)
GP-NAS Models				
GRU_R(TF(LSTM_R(ID)))	**155.06**	22.59	36.41	7.15
LSTM_R(TF(x))	155.78	**22.66**	**36.48**	7.05
GRU_R(ATT(TF(LSTM_R)))	161.99	22.35	35.99	7.22
Baseline Models (no GloVe, no POS)				
Plain GRU	202.39	21.57	33.59	10.29
Plain LSTM	240.42	20.39	31.97	10.36
Plain Transformer	293.29	18.42	30.29	10.49

Fig. 5 shows a visual comparison of test perplexity across all models.

The best GP architecture reduces test perplexity by 23.4% relative to the best baseline (155.06 vs. 202.39) while using approximately 32% fewer parameters (7.15M vs. 10.29M). This efficiency gain is visualized in Fig. 6.

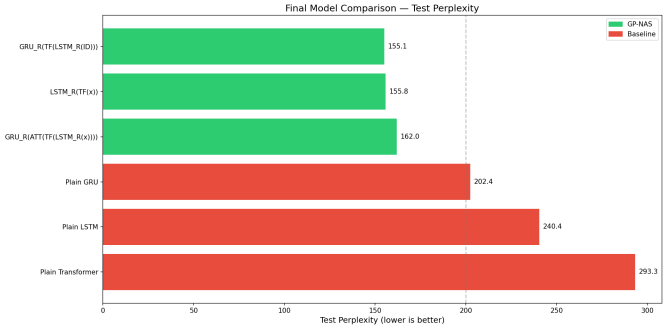

Fig. 5. Test perplexity comparison between GP-NAS architectures (green) and baselines (red). Lower is better. All GP-NAS models outperform all baselines.

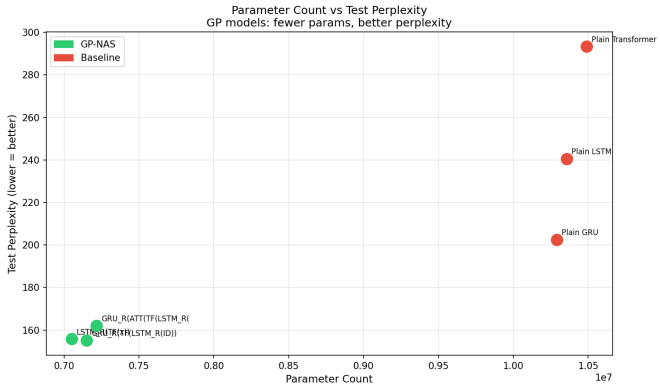

Fig. 6. Parameter count vs. test perplexity for all models. GP-NAS models cluster in the lower-left region: fewer parameters and better perplexity.

E. Training Dynamics

Fig. 7 show the training and validation curves for all three GP architectures during full training.

Training loss decreases steadily across all three architectures. There is a notable acceleration in convergence at epoch 4 when GloVe embeddings are unfrozen (Fig. 7), suggesting that fine-tuning the pretrained embeddings contributes meaningfully to the final performance. The validation perplexity curves show that best validation perplexity is achieved around epochs 5–7 for `LSTM_R(TF(x))` and `GRU_R(TF(LSTM_R(ID)))`, with early stopping triggered at epoch 9 for both. `GRU_R(ATT(TF(LSTM_R(x))))` trains for all 10 epochs due to its slower convergence.

Table III shows the best validation perplexity versus test

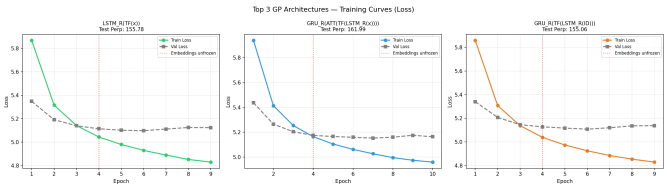

Fig. 7. Training and validation loss curves for the top 3 GP architectures. The vertical dotted line marks epoch 4, when GloVe embeddings are unfrozen.

perplexity for each architecture.

TABLE III
BEST VALIDATION VS. TEST PERPLEXITY — TOP 3 GP ARCHITECTURES

Architecture	Best Val PPL	Test PPL
LSTM_R(TF(x))	163.53	155.78
GRU_R(ATT(TF(LSTM_R)))	172.94	161.99
GRU_R(TF(LSTM_R(ID)))	165.33	155.06

F. Accuracy Comparison

Fig. 8 compares Top-1 and Top-3 accuracy across all models.

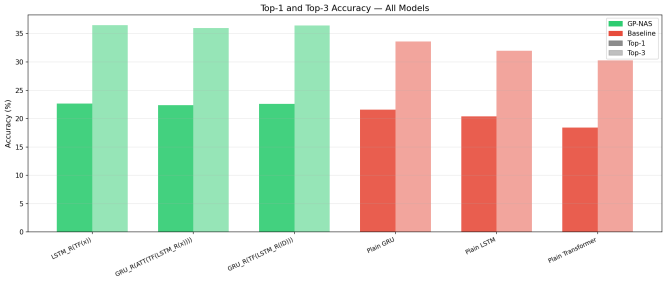

Fig. 8. Top-1 and Top-3 accuracy comparison across all models. GP-NAS models consistently outperform baselines on both metrics.

All three GP models exceed 22.3% Top-1 accuracy and 35.9% Top-3 accuracy. The best baseline (Plain GRU) achieves 21.57% and 33.59% respectively, confirming that the GP-NAS pipeline produces architectures with stronger predictive performance across all metrics.

VI. DISCUSSION

A. Why Hybrid Architectures Win

The LSTM-Transformer hybrid system maintains its excellence because it consistently achieves better results than other systems during evaluation tests. The pure recurrent architecture design processes sequences through a series of steps because its information processing method requires sequential operation. Pure Transformers process all time steps at once through self-attention but they lack the ability to recognize local sequential patterns which RNNs use as their inductive bias. The hybrid stack system `GRU_R(TF(LSTM_R(ID)))` uses two advantages because its recurrent layers efficiently capture short-term time relationships while its Transformer block delivers complete information through its attention system. The GP search process discovered hybrid structures through re-evaluation and complete training even though its proxy evaluation method failed to evaluate them.

The results show that deeper stacks produced no improvements. The system `GRU_R(ATT(TF(LSTM_R(x))))` which consists of four blocks represents the highest depth for top three systems but it recorded the lowest test perplexity score of 161.99 among the three test contenders. The research shows that at a hidden size of 128 system performance decreases when users apply additional deepness because two to three blocks already provide maximum capacity. The system performance reaches its peak at this point because users need to spend more time on optimization as they try to build complex systems.

B. The Proxy Evaluation Problem

The study proves that proxy rank and true rank show no connection to each other. The root cause shows that GRU and LSTM architectures function better when training lasts for short periods because their gating systems enable strong gradient movement from their first training session. The training process requires multiple educational periods because Transformer blocks need to develop their first attention methods, which operate at their base level throughout their initial educational sessions until they reach their full potential after several data review cycles. The evaluation process for Transformer systems through 2–3 epoch proxy assessment results in faulty results because the systems need time to demonstrate their abilities, which leads to their actual fitness assessment being lower than their real capabilities.

The tournament selection mechanism creates this situation because GRU-dominant individuals maintain their high proxy fitness levels, which enables them to win tournaments. The fitness curve (Fig. 2) shows that best fitness maintains its level after generation 2, while the best-tree field in `gp_run_summary.json` shows `GRU(GRU_R(LSTM_R(ID)))` winning every generation from 2 to 10. The random immigrant mechanism injects diversity every generation yet fails to stop the proxy from consistently undervaluing novel TF/ATT hybrid models.

The two-stage re-evaluation process served as the crucial method which restored the actual optimal architectural designs. The submitted designs would have included GRU stacks that performed worse than they actually did without this process.

C. Pipeline Comparison and Preprocessing Contribution

The GP-NAS models demonstrate a performance gap to baseline models because of two combined elements which include their architectural structure and their data processing methods. The GP-NAS pipeline uses GloVe 100d pretrained embeddings which achieve 96.2% vocabulary coverage, while the baselines apply randomly initialized embeddings together with their extensive but less common vocabulary system. The baseline systems lack this feature which enables POS tag embeddings to deliver syntactic information.

We compare these systems as full end-to-end systems because we want to test how architecture affects their performance. The preprocessing design is itself a deliberate choice in our system: we argue that a well-designed NAS pipeline should include input quality as a design dimension. A direct comparison of architectural elements would show how GP search improves model performance because baseline models use the same GloVe and POS features which all models share. This research topic needs investigation in future studies.

VII. Conclusion

The research introduced a Genetic Programming-based Neural Architecture Search system for next-word prediction on WikiText-2. The GP framework of our research developed sequence model architectures through a tree structure that linked different types of neural blocks and used two evaluation stages to find effective models which did not require complete training during the evaluation process.

The best architecture, `GRU_R(TF(LSTM_R(ID)))`, achieved a test perplexity of 155.06 — a 23.4% improvement over the best baseline — while using 32% fewer parameters. All top GP architectures consisted of LSTM-Transformer hybrids which became the dominant design choice after our team conducted additional evaluations of the system beyond its initial proxy-guided GP search testing.

The proxy evaluation system failed to deliver valid results because it produced a Spearman rank correlation of 0.021 between proxy and true rankings which proved that short-epoch evaluations on partial data are fundamentally biased against Transformer-containing architectures. The discovery directly affects all NAS systems which depend on proxy fitness assessment to direct their search process through various block types.

VIII. Future Work

The system requires multiple enhancements which will be implemented in its upcoming versions.

Proxy evaluation redesign: The primary requirement needs a solution which requires extended proxy assessments through 5 epochs on 40% data while utilizing a learning rate warmup that specifically meets Transformer block requirements. The alternative fitness metric should measure how fast the rate of perplexity improved between two points rather than assessing final perplexity because it allows better evaluation of different architectural models which reach their endpoint at various speeds.

GP grammar refinement: The `ID` terminal and standalone `DROP` node should be removed or constrained, as they contribute little architectural meaning. The system requires Feed-Forward blocks to appear only after users implement either recurrent or attention blocks which results in grammatically strong trees.

Capacity scaling: The system designer selected hidden size 128 as the operational boundary which they would use to control system performance. The system will achieve its objective through scaling to 256 which increases the Transformer feedforward dimension from 256 to 512 and implements 2-layer GRU/LSTM blocks to create a valid benchmark against existing language modeling standards.

Seeded immigrants and adaptive mutation: Random immigrants should include pre-seeded TF/ATT-containing architectures to prevent early population collapse. The mutation rate will increase when population fitness remains constant because this approach enables system diversity which leads to successful output.

Preprocessing ablation: The researchers will conduct their study by using GloVe+POS preprocessing to train baseline models while maintaining identical preprocessed data which will help them understand how architecture search affects performance. The process needs to separate components because it serves as the foundation for GP-NAS assessment.

Embedding unfreeze schedule: The system will achieve better fine-tuning performance when users unfreeze GloVe embeddings at epoch 7 instead of epoch 4 while they decrease learning rates to 10^{-4}.

References

[1] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," in *Proc. ICLR*, 2017.

[2] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," in *Proc. AAAI*, 2019.

[3] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," in *Proc. ICLR*, 2019.

[4] J. R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, 1992.

[5] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," in *Proc. ICLR*, 2017.

[6] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proc. EMNLP*, 2014.

[7] F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau, and C. Gagné, "DEAP: Evolutionary algorithms made easy," *Journal of Machine Learning Research*, vol. 13, pp. 2171–2175, 2012.

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. NeurIPS*, 2017.

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[10] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. EMNLP*, 2014.

[11] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, "A neural probabilistic language model," *Journal of Machine Learning Research*, vol. 3, pp. 1137–1155, 2003.

[12] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. ICLR*, 2015.

[13] P. J. Angeline, G. M. Saunders, and J. B. Pollack, "An evolutionary algorithm that constructs recurrent neural networks," *IEEE Transactions on Neural Networks*, vol. 5, no. 1, pp. 54–65, 1994.

[14] S. R. Young, D. C. Rose, T. P. Karnowski, S.-H. Lim, and R. M. Patton, "Optimizing deep learning hyper-parameters through an evolutionary algorithm," in *Proc. Workshop on Machine Learning in High-Performance Computing Environments*, 2015.

[15] H. Schmid, "Probabilistic part-of-speech tagging using decision trees," in *Proc. International Conference on New Methods in Language Processing*, 1994.

[16] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019.

[17] T. Wolf et al., "HuggingFace's Transformers: State-of-the-art natural language processing," in *Proc. EMNLP (Systems Demonstrations)*, 2020.

[18] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[19] J. L. Ba, J. R. Kruskal, and R. Salakhutdinov, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[20] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.